

Chapter:7 Input/Output

Prepared By Er Abhishek Singh
Computer Engineer
Department Of IT
Government of Nepal

Chapter#7 Input-Output Organization

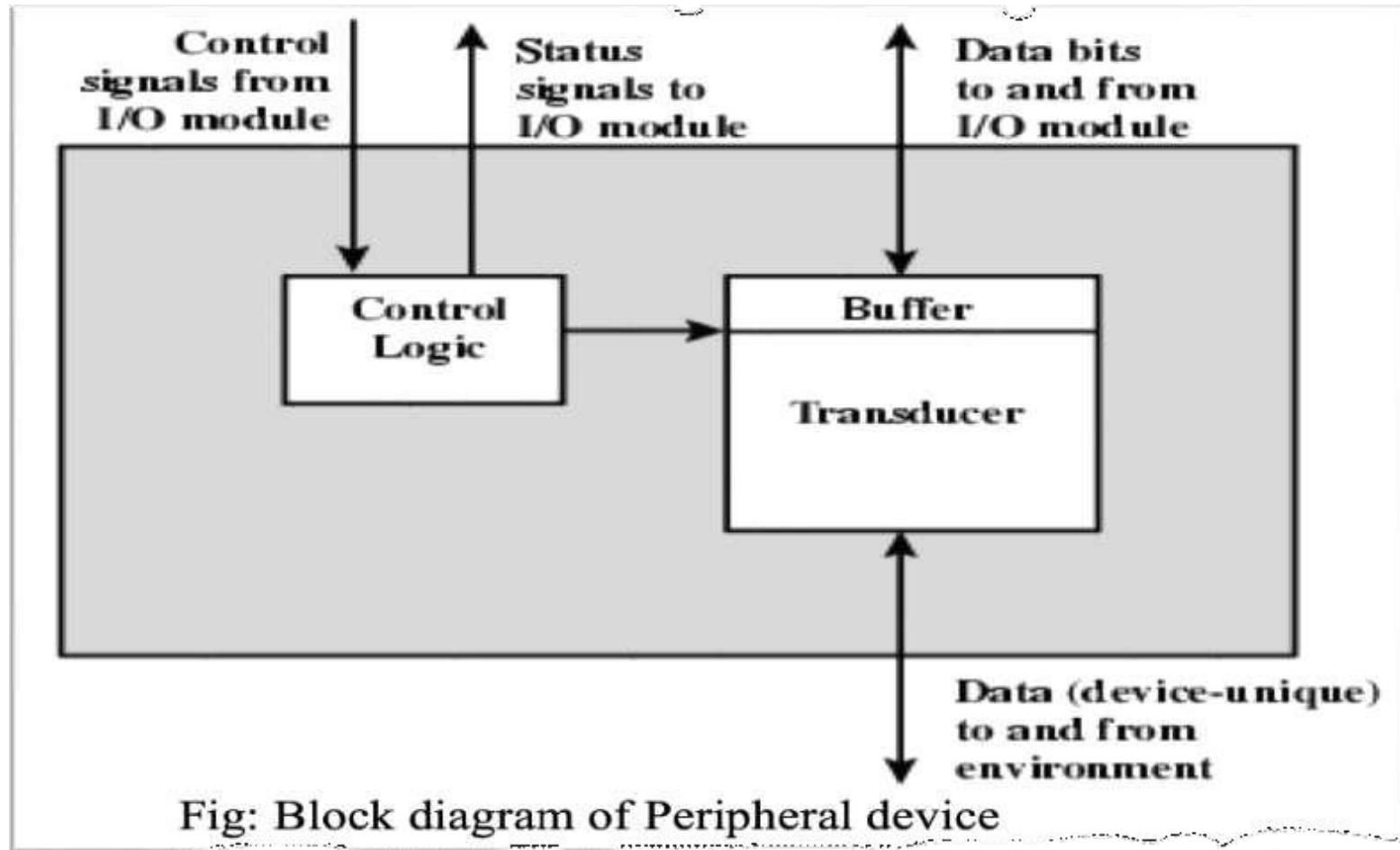
- Peripheral Devices
- I/O Modules
- Input-Output Interface
- Modes of Transfer
 - Programmed I/O
 - Interrupt-driven I/O
 - Direct-Memory I/O
- I/O Processor
- Data Communication Processor

Peripheral Devices

Input or output devices attached to the computer are also called peripherals.

- Among the most common peripherals are keyboards, display units, and printers.
- A peripheral device is any external device that connects to a computer to provide input, output, or storage functions, but is not part of the core CPU or memory. These devices allow the computer to interact with the outside world and extend its capabilities.
- Perhaps those provide auxiliary storage for the systems are magnetic disks and tapes.
- We can broadly classify peripheral devices into three categories:
 - Human Readable: Communicating with the computer users, e.g. display unit, monitor, Keyboard, mouse, scanner, printers etc.
 - Machine Readable: Communicating with equipment's, e.g. HDD, CD-ROM, Flash Drive, SD card,
 - Communication: Communicating with computer machine or remote devices as a means of data exchange, e.g. modem, NIC card etc.

Peripheral Devices



1. Control Logic

- Function: Acts as the “brain” of the peripheral device.
- Inputs:
 - Receives Control Signals from the I/O module (part of the CPU or system bus).
- Outputs:
 - Sends Status Signals back to the I/O module (e.g., device ready, busy, error).
 - Sends control commands to the internal Buffer/Transducer unit.
- Role: Interprets control commands, initiates device-specific operations, and manages data flow within the peripheral.

2. Buffer

- Function: Temporary storage area for data.
- Bidirectional Flow:
 - Holds data bits to and from the I/O module.
 - Acts as a staging area between the CPU/system and the transducer.
- Importance: Allows for speed mismatches between the I/O device and the processor or memory.

3. Transducer

- Function: Converts physical signals to digital data or vice versa.
 - Input device example: Microphone (sound → electrical signals).
 - Output device example: Printer (digital data → printed text).
- Role: Interface between the physical world and the digital system.
- Data Flow: Sends/receives device-unique data to/from the external environment.

4. External Signals

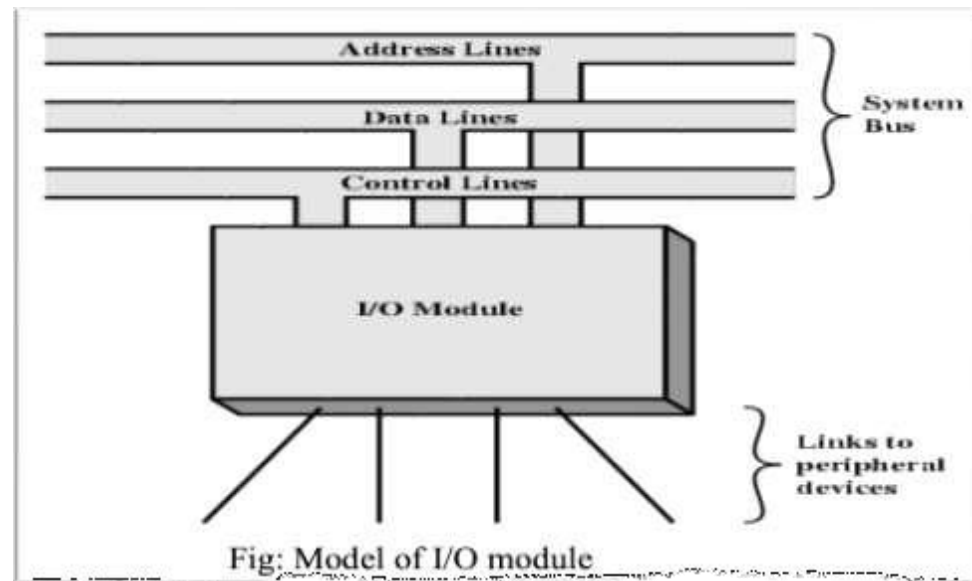
- Control Signals (from I/O module): Command signals such as read, write, or device enable.
- Status Signals (to I/O module): Indicates current device status (e.g., idle, busy, error).
- Data Bits (to/from I/O module): Actual information to be processed or received.
- Data (to/from environment): Real-world input/output handled by the transducer.

Working Summary:

1. CPU sends control signals via the I/O module.
2. Control Logic interprets these signals and directs the buffer/transducer.
3. The transducer performs data conversion as per the peripheral function.
4. Buffer temporarily stores or prepares data for exchange.
5. Data bits are exchanged between buffer and I/O module.
6. Status signals are sent to the CPU for coordination and error handling.

I/O Module

- I/O modules interface to the system bus or central switch (CPU and Memory), and provides interfaces and controls to one or more peripheral devices.
- I/O operations are accomplished through a wide assortment of external devices that provide a means of exchanging data between external environment and computer by a link to an I/O module.
- The link is used to exchange control status and data between I/O module and the external devices.



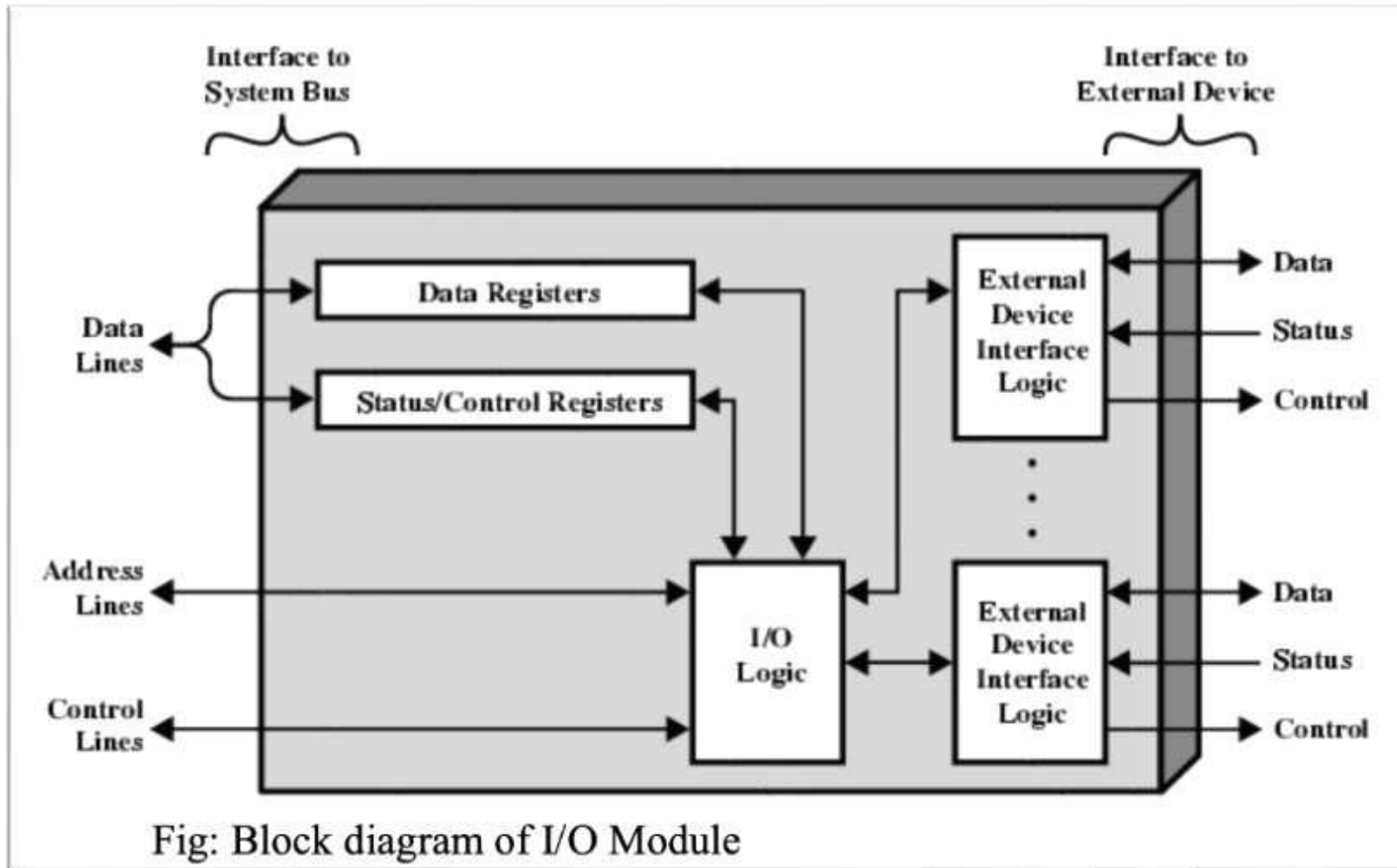
I/O Module

- Peripherals are not directly connected to the system bus instead an I/O module is used which contains logic for performing a communication between the peripherals and the system bus.
 - There are a wide variety of peripherals with various methods of operation.
 - The data transfer rate of peripherals is often much slower than that of the memory or processor.
 - Peripherals often use different data format and word length than the computer to which they are connected.

I/O Module Functions

- Timing and Control:
 - I/O module includes control and timing to coordinate the flow of traffic between internal resources and external devices.
- Processor Communication
 - Command decoding: I/O module accepts commands from the processor.
 - Data: Data are exchanged between the processor and I/O module over the bus.
 - Status reporting: Peripherals are too slow and it is important to know the status of I/O module.
 - Address recognition: I/O module must recognize one unique address for each peripheral it controls.
- Device Communication:
 - Data Buffering (to synchronize device and memory speed)
 - Error Detection (bit error or signal error or machine error)

I/O Module Structure



The I/O module manages and controls data transfer between CPU and external devices, while its internal registers store data/status and its interface logic connects to specific devices.

1. System bus interface

This part connects the module to the CPU.

Data lines → Data registers

Carry actual data between CPU and I/O module.

Data registers temporarily store incoming or outgoing data (buffering).

Address lines → I/O logic

Used by CPU to select this I/O module or one of its registers/devices.

Control lines → I/O logic

Carry commands like read, write, interrupt acknowledge, etc.

These signals tell the module what operation to perform.

Function:

Allows CPU to communicate with the I/O module through standard bus signals.

2. Internal registers

Inside the module are two main register groups.

Data registers

Hold data being transferred

Act as buffer between fast CPU and slow device

Function: Temporary storage for data transfer.

Status/Control registers

Status part: tells CPU whether device is ready, busy, or has error

Control part: stores commands from CPU (start, stop, read, write)

Function:

Provide communication about device condition and control instructions.

ingh

3. I/O Logic (controller)

This is the central controller of the module.

Functions:

- Interprets address and control signals from CPU

- Controls when data moves between registers and device

- Generates control signals for devices

- Handles interrupts

- Synchronizes speed between CPU and device

It coordinates all operations inside the I/O module.

4. External device interface logic (right side)

Each external device has its own interface circuit.

Function:

Converts signals between I/O module and device

Handles device-specific timing and format

Sends/receives:

- Data signals

- Status signals

- Control signals

This ensures the module can communicate with different types of devices.

Overall working

CPU sends command and address via system bus

I/O logic selects device and operation

Data moves through data registers

External interface communicates with device

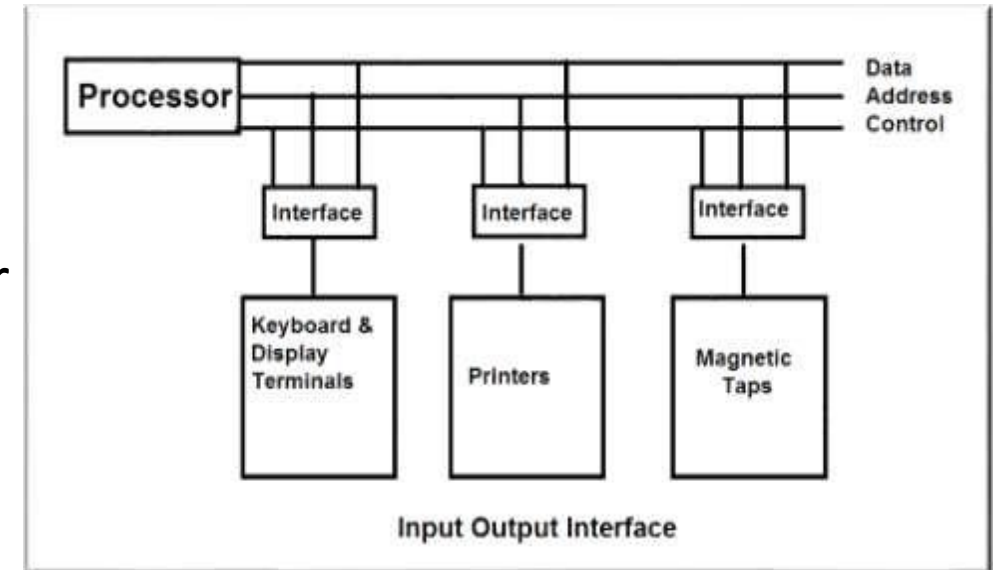
Status is updated and sent back to CPU

Input-Output Interface

- Input-Output interface provides a method for transferring information between internal storage (such as memory and CPU registers) and external I/O devices.
- Peripherals connected to a computer need special communication links for interfacing them with the central processing unit.
- The communication link (the interface) resolves the following differences between the computer and peripheral devices.
 - Devices and signals (Peripherals - Electromechanical Devices; CPU or Memory - Electronic Device)
 - Data Transfer Rate (Peripherals - Usually slower; CPU or Memory - Usually faster than peripherals)
 - Unit of Information (Peripherals – Byte; CPU or Memory – Word)

Input-Output Interface

- The Interface performs:
 - Synchronizes the data flow and supervises the transfer rate between peripheral and CPU or Memory
 - Decodes the device address (device code)
 - Decodes the commands (operation)
 - Provides signals for the peripheral controller



Memory Mapped I/O and Isolated I/O

- CPU needs to communicate with the various memory and input-output devices (I/O). Data between the processor and these devices flow with the help of the system bus. There are three ways in which system bus can be allotted to them:
- Separate set of address, control and data bus to I/O and memory.
- Have common bus (data and address) for I/O and memory but separate control lines.
- Have common bus (data, address, and control) for I/O and memory.

In first case it is simple because both have different set of address space and instruction but it require more buses.

Memory Mapped I/O and Isolated I/O

Isolated I/O

In Isolated I/O, the CPU uses the same buses (wires) to talk to both memory and I/O devices, but it has separate control signals to tell whether it's dealing with memory or an I/O device.

I/O devices have special addresses called ports.

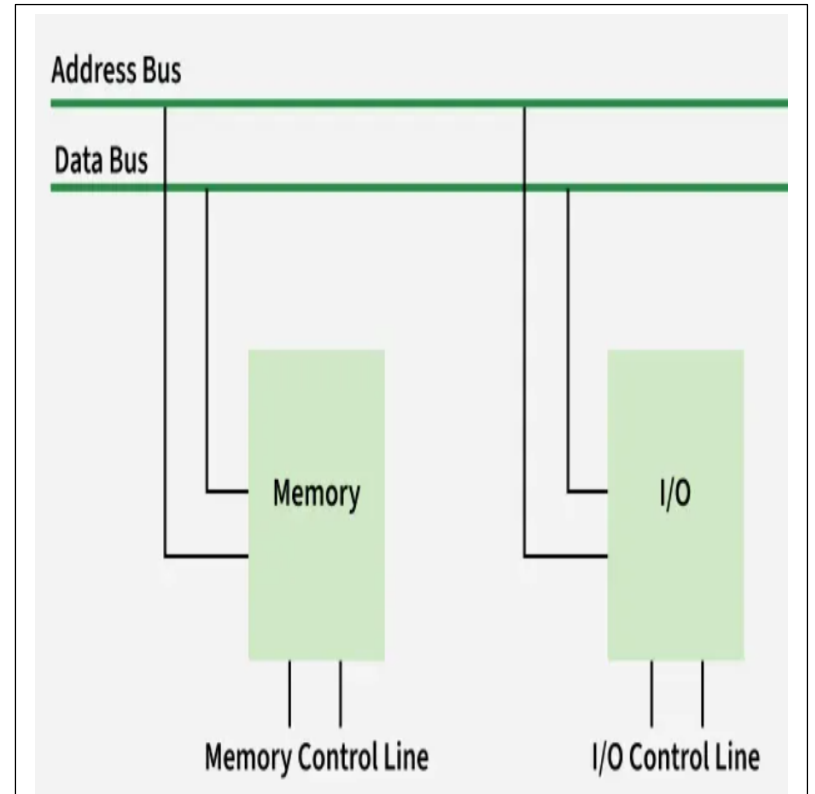
When the CPU wants to communicate with an I/O device:

It puts the port address on the address bus.

It uses special control lines like I/O Read or I/O Write.

Then data is sent or received using the data bus.

As memory and I/O have separate address spaces, it's called Isolated I/O. Also, the CPU uses different instructions for memory and I/O (like IN and OUT for I/O).



Memory Mapped I/O

In a memory-mapped I/O system, there are no special input or output instructions. Instead, the CPU uses the same instructions it uses for memory (like LOAD and STORE) to access I/O devices.

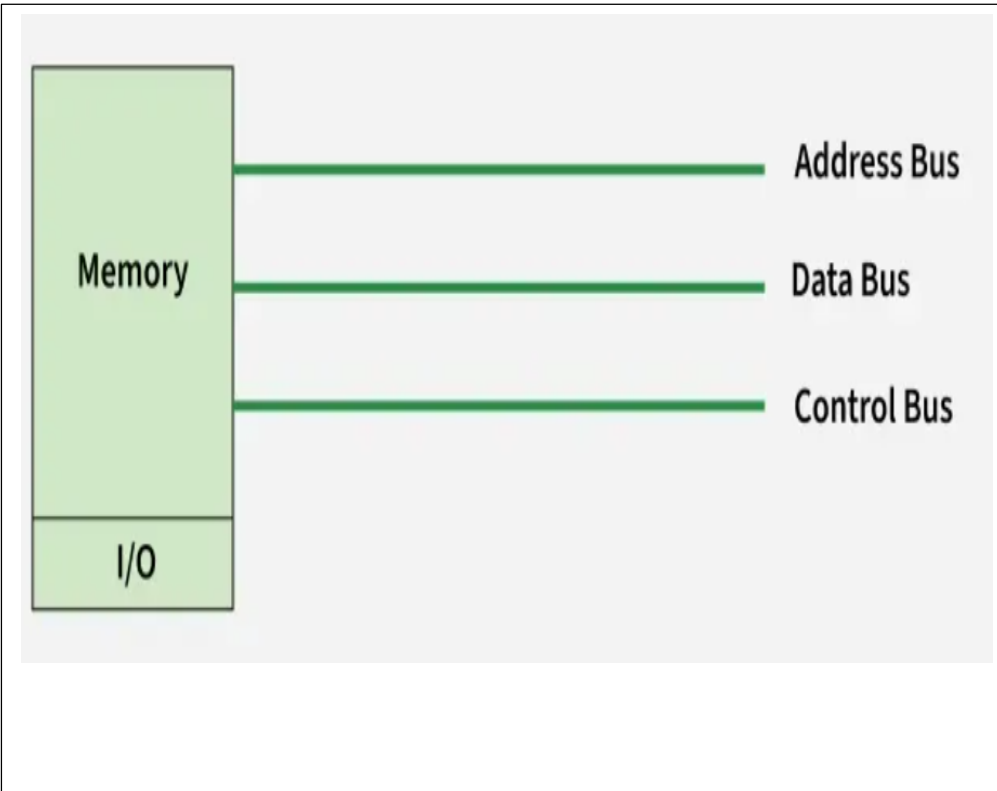
Each I/O device is assigned a specific address in the regular memory address space.

Devices are connected through interface registers, which act like memory locations.

When the CPU wants to read from or write to an I/O device, it accesses the corresponding address, just like it would access a memory word.

These interface registers respond to normal read/write operations as if they were memory cells.

This design allows I/O and memory to be treated uniformly, simplifying programming and hardware design.



Isolated I/O versus Memory Mapped I/O

Isolated(I/O mapped) I/O

- Separate I/O read/write control lines in addition to memory read/write control lines
- Separate (isolated) memory and I/O address spaces
- Distinct input and output instructions

Memory mapped I/O

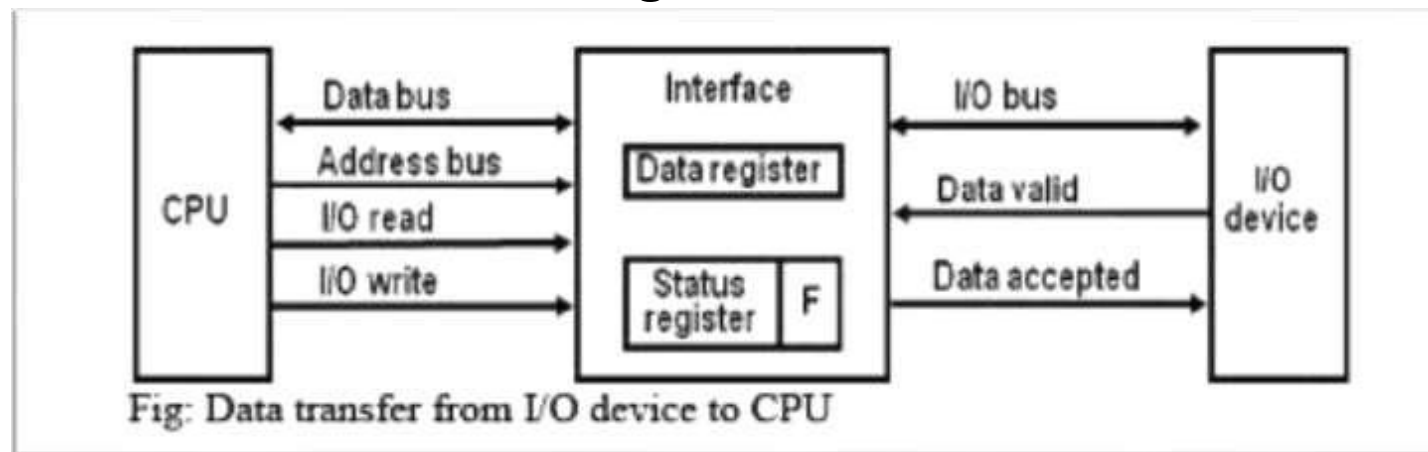
- A single set of read/write control lines (no distinction between memory and I/O transfer)
- Memory and I/O addresses share the common address space which reduces memory address range available
- No specific input or output instruction so the same memory reference instructions can be used for I/O transfers
- Considerable flexibility in handling I/O operations

Mode of Transfer

- Data Transfer between the central computer and I/O devices may be handled in a variety of modes.
- Some modes use CPU as an intermediate path, others transfer the data directly to and from the memory unit.
- Data transfer to and from peripherals may be handled in one of three possible modes.
 - Programmed I/O
 - Interrupt Driven I/O
 - Direct Memory Access (DMA)

Programmed I/O

- Programmed I/O generally used to transfer data to/ from I/O devices (such as keyboard or monitor) to/ from memory with the supervision of CPU.
- As name suggests, It is implemented with the help of program instruction such as IN, OUT, STORE.
- As we already know that to compensate differences (in transfer rate, data format, variety of devices etc.) between processor and I/O devices we deploy I/O interface (or I/O module in between as shown in figure below.

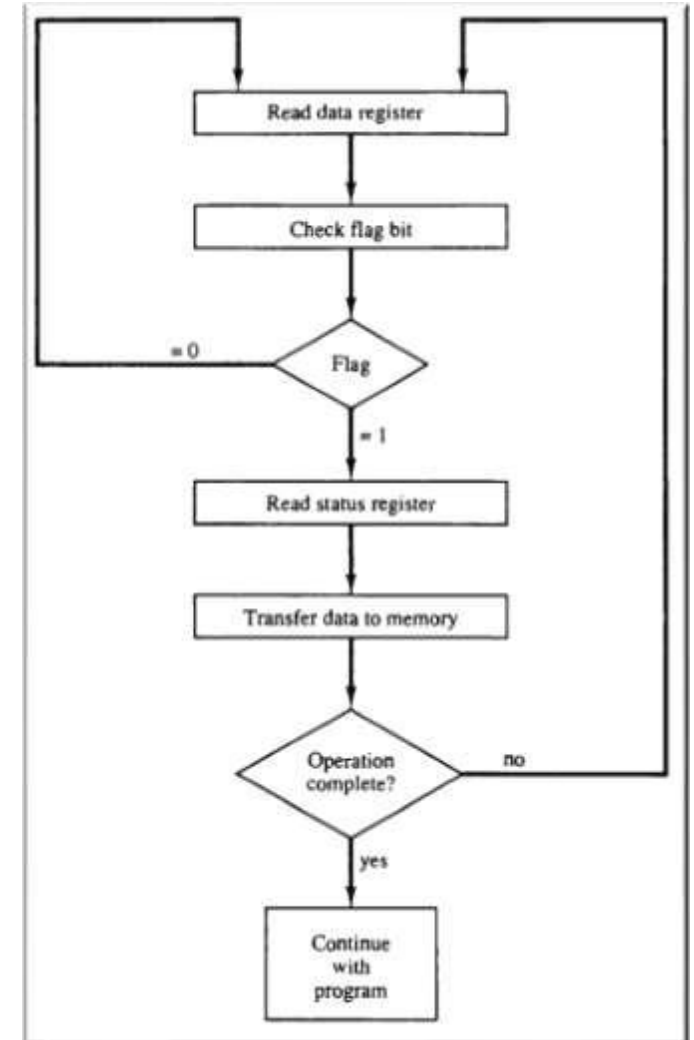


Programmed I/O

- Lets see above figure:
- I/O devices places its data to ***I/O Bus*** and enables ***Data Valid*** line
- The Data Valid line specifies that some information is in I/O Bus
- Now the data from I/O bus will present in ***Data Register*** of I/O Interface
- Once it holds data on the register, the Flag bit (***F***) will set to 1 and the interface will enable ***Data Accepted*** (like ACK) line
- In between CPU and Interface, there are three buses [Data, Address, Control (I/O Read, I/O Write)]
- Now, CPU checks F bit all the time; once it set to 1, CPU does its job to bring the data from Data Register to Memory.

Programmed I/O

- Flow Chart (CPU perspective)
- If flag bit is 1, data transfer takes place
- If flag bit is 0, CPU again check the status bit
- TOO MUCH CPU UTILIZATION (NO GOOD)
- It's a responsibility of CPU to check the status Register all the time



Advantages of Programmed I/O

- . Provides simple and easy interfaces to programs.
- . Direct access to interact with the CPU without involving any peripheral devices in the process.

Disadvantages of Programmed I/O

- . Synchronization is not efficient in terms of the CPU time as it will need to loop waiting for the device to be ready.
- . The CPU is idle and cannot handle any other task at the time it waits for the I/O device to complete processing.

Interrupt-driven I/O

- **What is an Interrupt?**

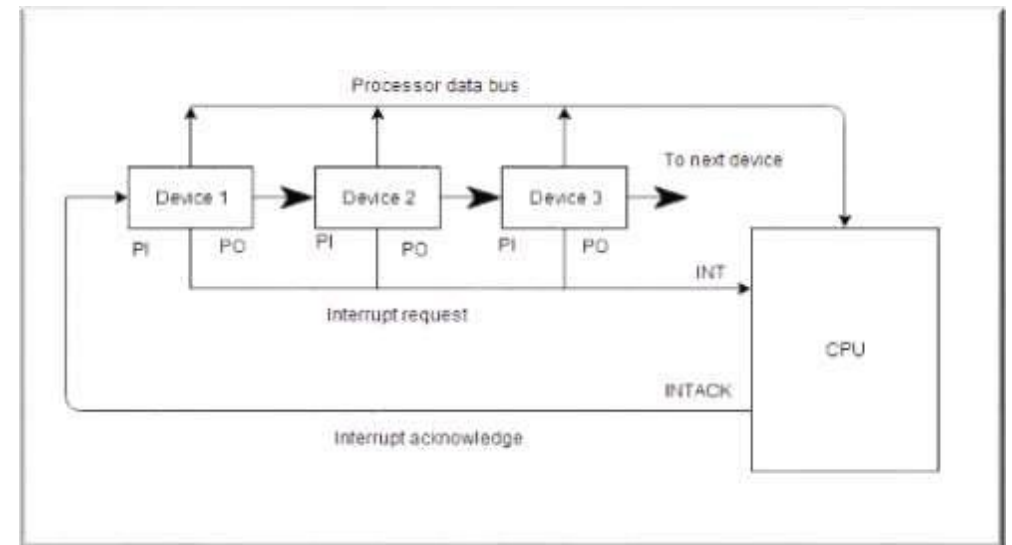
- An alternative to the CPU constantly monitoring the flag is to let the interface inform the computer when it is ready to transfer data
- This mode of transfer uses the interrupt facility. While the CPU is running a program, it does not check the flag.
- However, when the flag is set, the computer is momentarily interrupted from proceeding with the current program and is informed of the fact that the flag has been set
- The CPU deviates from what it is doing to take care of the input or output transfer
- After the transfer is completed, the computer returns to the previous program to continue what it was doing before the interrupt
- The CPU responds to the interrupt signal by storing the return address from the program counter into a memory stack and then control branches to a service routine that processes the required I/O transfer
- The way that the processor chooses the branch address of the service routine varies from one unit to another.

Interrupt-driven I/O

- Here, the main objective of this mode is to utilizing the CPU efficiency in effective manner
- It is the responsibility of I/O interface to sends interrupt signal to the CPU (so that CPU can be busy with other tasks)
- And, whenever CPU receives interrupt signal than only it take cares of the interrupt handling (not go to check every time whether the I/O device is trying to communicate with CPU or not)
- However there is a big issue that multiple devices are sending interrupt(s) simultaneously.
- And, so which interrupt should be executed first??? [*Priority Interrupt*]

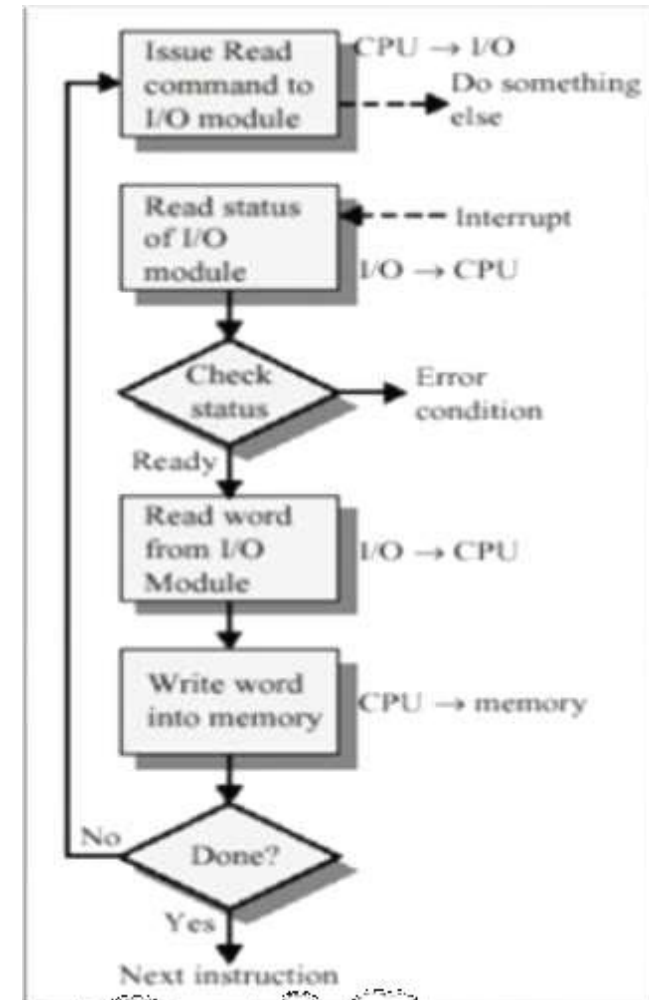
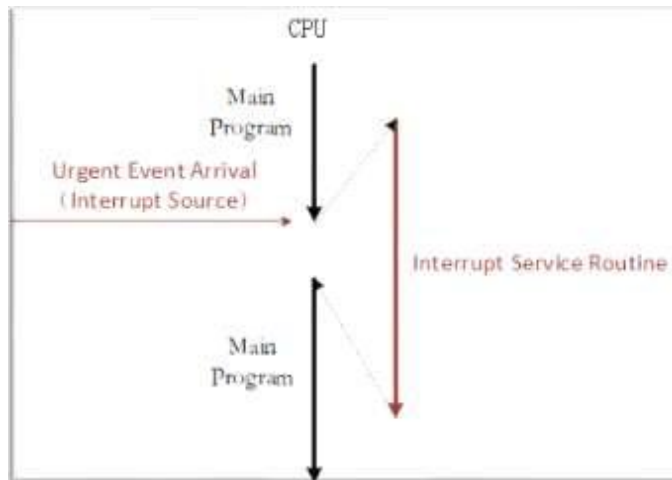
Priority Interrupt

- The I/O device having highest priority (0) executes first
- TWO types of Solution(s):
- Software approach
 - Polling
 - Program check whether the highest priority device issued the interrupt or not If not Checks for the lower priority interrupts in the order
- Hardware approach
 - Daisy Chain
 - Same as Pooling but in hardware mode
 - (See figure)
 - Parallel Priority Interrupt (Self Study?)



Interrupt-driven I/O (Flow Chart)

- No need to wait CPU for I/O module
- When the I/O module is ready, it interrupts CPU
- Once CPU receives interrupt:
- Uses Interrupt Handler and calls ISR
- After completion of ISR:
- It goes back to the main program



Advantages of Interrupt-Initiated I/O

- Reduced overhead on the CPU as the same CPU waits for an interrupt while doing other tasks at the same instance.
- There is also no need for monitoring the status of a specific device.
- Reduces CPU time wastage, hence enhancing the performance of the system.

Disadvantages of Interrupt-Initiated I/O

- Difficult to implement and program in contrast to sequential file processing or compared to it when it is implemented in low-level languages.
- Needs interrupt handling routines, which can be time-consuming.

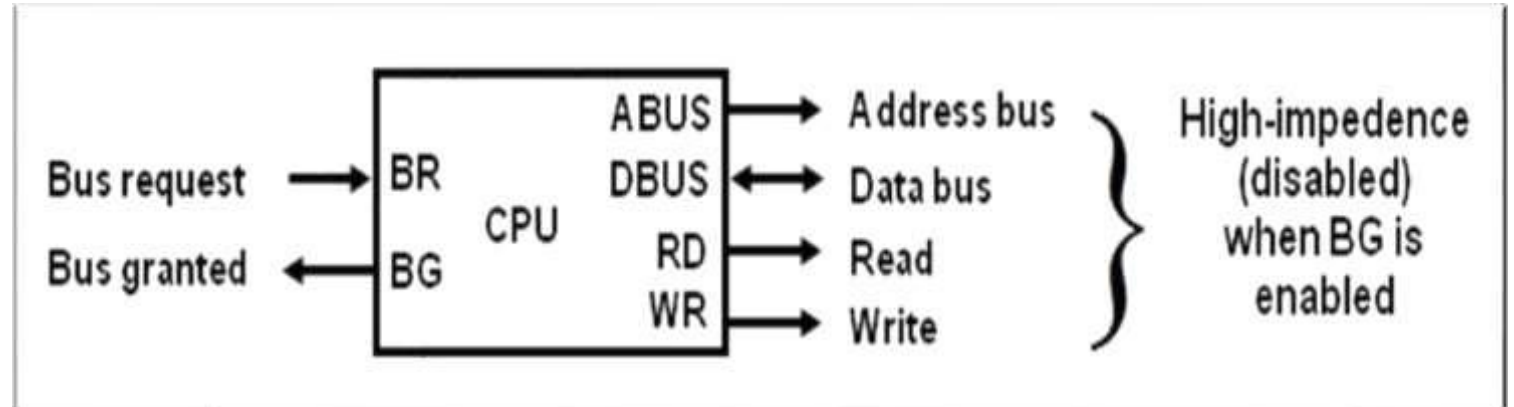
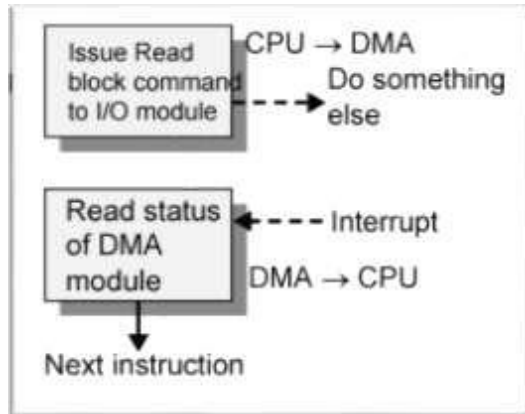
Programmed I/O	Interrupt Initiated I/O
Data transfer is initiated by the means of instructions stored in the computer program. Whenever there is a request for I/O transfer the instructions are executed from the program.	The I/O transfer is initiated by the interrupt command issued to the CPU.
The CPU stays in the loop to know if the device is ready for transfer and has to continuously monitor the peripheral device.	There is no need for the CPU to stay in the loop as the interrupt command interrupts the CPU when the device is ready for data transfer.
This leads to the wastage of CPU cycles as CPU remains busy needlessly and thus the efficiency of system gets reduced.	The CPU cycles are not wasted as CPU continues with other work during this time and hence this method is more efficient.
CPU cannot do any work until the transfer is complete as it has to stay in the loop to continuously monitor the peripheral device.	CPU can do any other work until it is interrupted by the command indicating the readiness of device for data transfer
Its module is treated as a slow module.	Its module is faster than programmed I/O module.

DMA (Direct Memory Access)

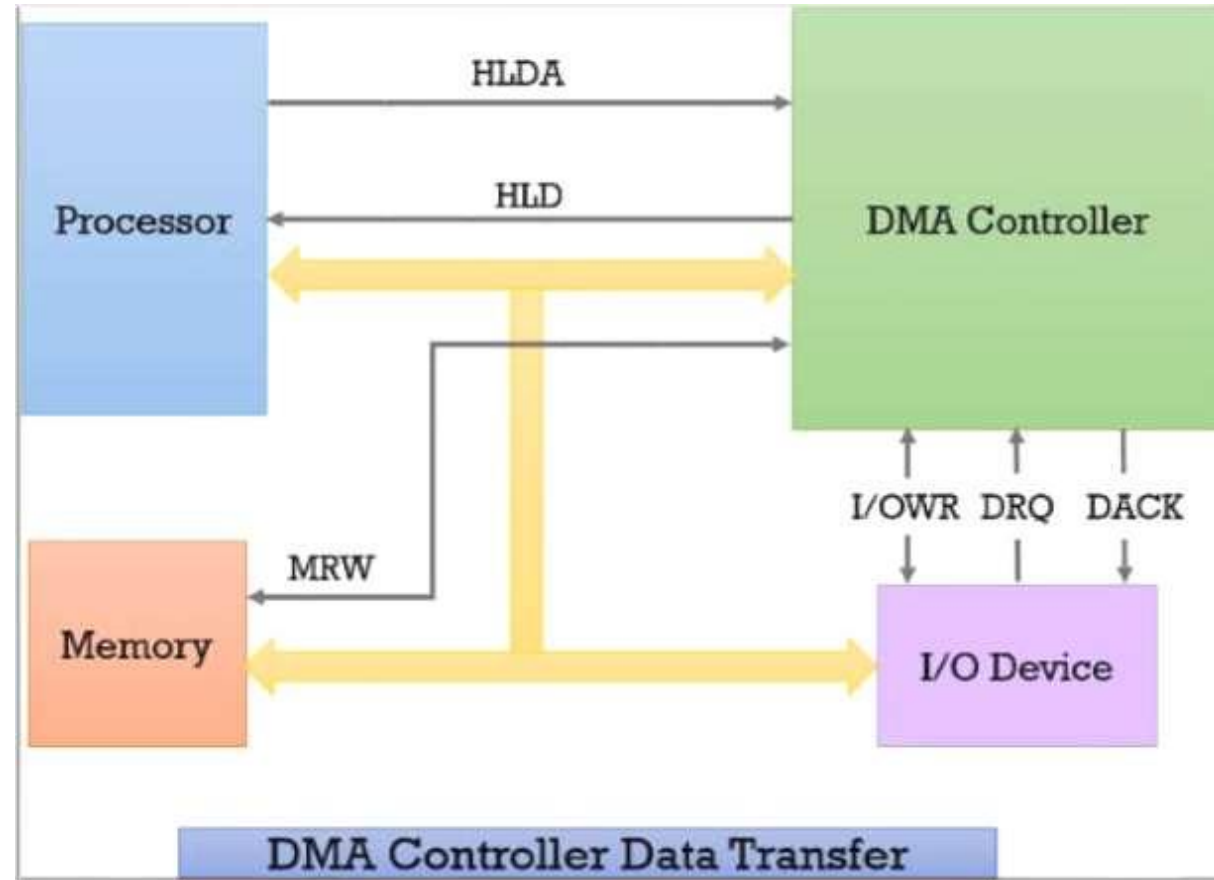
- Large blocks of data transferred at a high speed to or from high speed devices, magnetic drums, disks, tapes, etc.
- DMA controller Interface that provides I/O transfer of data directly to and from the memory and the I/O device
- CPU initializes the DMA controller by sending a memory address and the number of words to be transferred
- Actual transfer of data is done directly between the device and memory through DMA controller -> **Freeing CPU for other tasks**

DMA (Direct Memory Access)

- The transfer of data between the peripheral and memory without the interaction of CPU and letting the peripheral device manage the memory bus directly is termed as Direct Memory Access (DMA).
- The two control signals Bus Request and Bus Grant are used to facilitate the DMA transfer.
- The transfer of data between the memory and I/O are of two types:
 - **DMA Burst:** The block of data consisting a number of memory words is transferred at a time.
 - **Cycle Stealing:** DMA transfers one data word at a time after which it must return control of the buses to the CPU.



DMA Controller

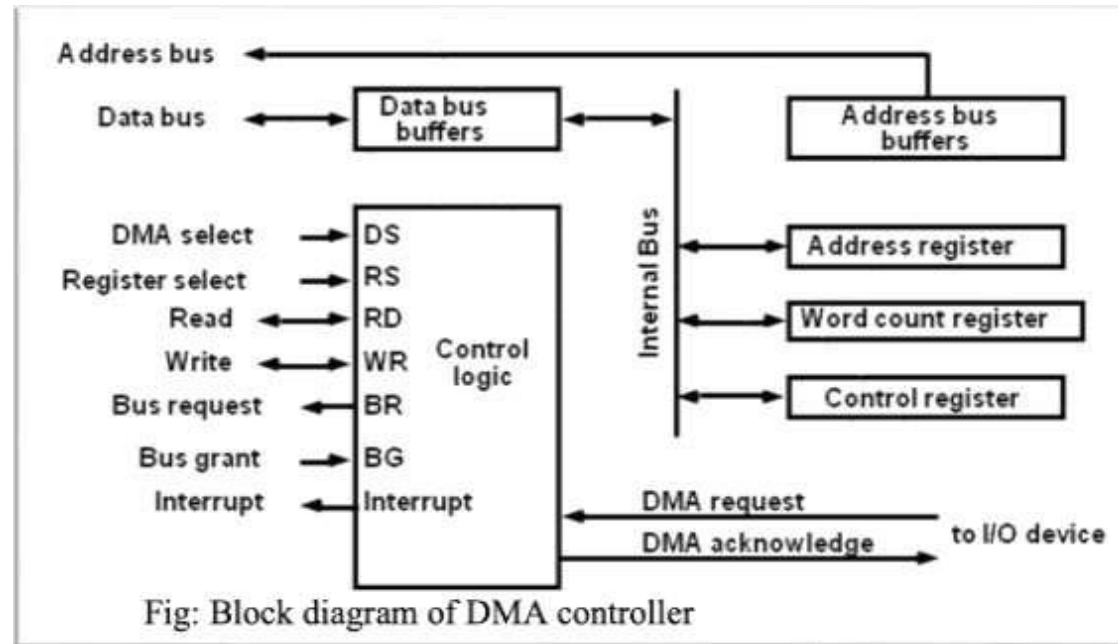


DMA Controller

- The DMA controller communicates with the CPU through the data bus and control lines.
- DMA select signal is used for selecting the controller, the register select is for selecting the register.
- When the bus grant signal is zero, the CPU communicates through the data bus to read or write into the DMA register.
- When bus grant is one, the DMA controller takes the control of buses and transfers the data between the memory and I/O.
- (See Figure)

DMA Controller

- The address register specifies the desired location of the memory which is incremented after each word is transferred to the memory.
- The word count register holds the number of words to be transferred which is decremented after each transfer until it is zero.
- When it is zero, it indicates the end of transfer (bus grant signal goes low; CPU takes control)



I/O Processor

It is a processor with direct memory access capability that communicates with IO devices.

IOP is similar to CPU except that it is designed to handle the details of IO operation.

Unlike DMA which is initialized by CPU, IOP can fetch and execute its own instructions.

IOP instruction are specially designed to handle IO operation.

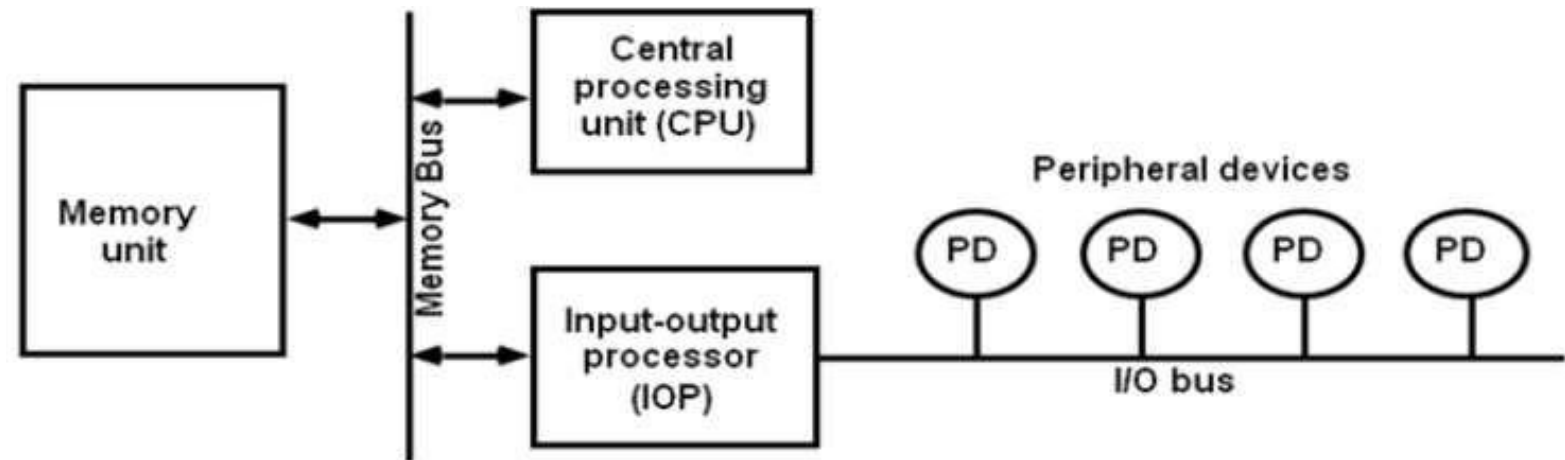


Fig: Block diagram of a computer with I/O Processor

Input-Output Processor (IOP)

An Input-Output Processor (IOP) is a specialized processor designed to handle all input/output (I/O) operations in a computer system, offloading the main CPU.

An IOP acts as a dedicated processor for I/O, managing all communication with peripheral devices and memory independently of the CPU, enabling faster and more efficient system operation

1. Purpose:

Allows the CPU to focus on computation while the IOP manages data transfer between peripherals and memory.
Reduces CPU idle time during slow I/O operations.

2. Working Theory:

IOP has its own control unit, registers, and sometimes small memory.

CPU sends commands to the IOP specifying the operation (read/write) and device address.

IOP performs the operation independently, handling data transfer between peripheral device ↔ memory, using techniques like DMA.

Once the operation is complete, IOP interrupts the CPU to signal completion or error.

1. CPU is fast, I/O is slow

The CPU operates at very high speeds (GHz), but I/O devices like disks, printers, keyboards, and network adapters are much slower.
If the CPU directly handles I/O, it spends a lot of time waiting for slow devices → inefficient use of CPU cycles.

2. Offloading I/O tasks

IOP can handle all data transfers, device control, and status checking independently.
CPU only sends a command to IOP and continues computation.

3. Parallelism

With IOP, CPU and I/O can work simultaneously: CPU executes programs
IOP transfers data in the background This improves overall system throughput.

4. Complex I/O management

I/O involves handshaking, buffering, error checking, and multiple devices.

Data Communication Processor

- The processor which distributes and collects data from many remote terminals connected through telephone and other communication lines is termed as Data Communication Processor (DCP)
- **It is responsible for:**
- Transmission Module:
 - Synchronous
 - Asynchronous
- Transmission Type:
 - Parallel
 - Serial
- Transmission Error Check:
 - Parity
 - Checksum
 - Cyclic Redundancy Check
- Transmission Modes:
 - Simplex
 - Half Duplex
 - Full Duplex
- Data Link & Protocol

A Data Communication Processor (DCP) is a specialized processor dedicated to managing data transmission, reception, and communication control between a computer system and external devices (like terminals, networks, modems, etc.).

It is similar in concept to an I/O processor, but specifically optimized for communication tasks.

Purpose of Data Communication Processor:

- Reduces CPU Burden
- Communication tasks involve:
 - Data formatting
 - Error detection and correction
 - Protocol handling
 - Synchronization
 - Buffer management

If the main CPU handles all these tasks, it wastes processing time.

A DCP offloads these operations, allowing the CPU to focus on computation.

- Handles Multiple Communication Lines
- In large systems:
 - Multiple terminals
 - Multiple network channels
 - Serial and parallel links

A DCP can manage many communication lines simultaneously using interrupts and buffering.

Improves System Performance

Without DCP:

- CPU waits for slow I/O devices
- System throughput decreases

With DCP:

- Parallel processing occurs
- Communication and computation happen simultaneously
- Overall performance increases

Protocol Handling

- Communication requires protocols such as:
 - Error checking (Parity, CRC)
 - Flow control
 - Packet assembly/disassembly

DCP implements these protocols in hardware or firmware, making communication faster and more reliable.

Data Communication Processor (DCP) is used in I/O organization to handle communication-related tasks such as protocol management, error control, buffering, and synchronization. It reduces the load on the main CPU by managing multiple communication lines independently. This improves system performance, increases throughput, and enables parallel execution of computation and communication tasks.

External Interface:

The external communication interface is used to provide an interface between the peripheral devices and I/O interface. There exist two main categories of external communication interfaces:

(a) Serial Communication Interface

(b) Parallel Communication Interface

The serial communication interface employs only one line for transferring one bit of data at a time.

The parallel communication interface can transfer several bits at the same time. The parallel Interfaces are usually utilized for I/O devices high-speed. For instance, disks and tapes use parallel interfaces. The communication that takes place across the communication interface includes the exchange of data and control information.

In both serial and parallel communication, the I/O interface must engage in communication with the peripheral. The communication steps for the read/write operation are given as follows:

- The I/O interface sends a control signal to I/O device to request permission to receive or send data.
- The I/O device accepts the Read/Write request and sends an acknowledgement.
- The data transfer operation either from I/O device to I/O interface (Read operation) or from I/O interface to I/O device (Write operation) is performed. The I/O device sends an acknowledgement signal after the data transfer operation.

An external interface is the communication link between a computer system and external devices (keyboard, printer, storage, network, etc.).

It defines how data, control, and signals travel between the system and peripherals.

External interfaces are mainly classified by connection method and working theory.

1. Control signal — Determines which operation the device will perform (e.g., READ or WRITE).
2. Status signals — Indicate the device's current state (e.g., READY or NOT-READY).
3. Data module — Manages the data bits to be sent or received.
4. Control logic — Executes device operations based on commands from the I/O system.
5. Transducer — Converts data between electrical signals and other forms of energy (e.g., during output: electrical → sound/light; during input: sound/light → electrical).

This is essentially describing the interface unit that handles communication between the CPU and an I/O device, covering control, status, data handling, and signal conversion.

Types of external interfaces:

1. Point-to-point
2. Multipoint

1. Point-to-Point Interface

A point-to-point interface connects exactly one sender and one receiver with a dedicated communication path.

In a point-to-point configuration, there is a direct, dedicated physical link between exactly two entities sharing the communication channel.

(e.g., a keyboard plugged into a dedicated port)

1. Dedicated Channel: Only two devices communicate directly with each other (e.g., a CPU and a specific peripheral, or two network switches).
2. High Speed and Efficiency: Because the bandwidth is not shared, this configuration avoids contention issues, leading to faster and more efficient data transfer
3. Simplicity in Management: It is simpler to manage as there is no need for complex addressing or arbitration schemes to decide which device can use the bus at any given time
4. Examples:
 - a. PCI Express (PCIe): Modern PCIe lanes are point-to-point connections between a host (CPU or chipset) and a device (GPU, SSD, etc.)
 - b. SATA: This interface connects a single drive to a host controller .

2. Multipoint Interface

- a) A multipoint interface allows multiple devices to share the same communication medium and it behaves like an external bus.
- b) Many devices connected to same line/bus and Only one device transmits at a time.
- c) Others listen.
- d) Requires addressing and arbitration.
- e) A single I/O module connects to multiple devices, often using a shared bus, allowing one interface to manage several peripherals (e.g., SCSI or USB hubs).]
- f) Multiple peripheral interfaces are attached to a common I/O bus. The processor places a device address on the bus to identify which peripheral it is communicating with.

Small Computer System Interface (SCSI)

- Small Computer System Interface (SCSI) is a high-performance multipoint external interface used to connect multiple peripheral devices (like hard disks, scanners, tape drives) to a computer through a common bus.
- It works on a shared bus architecture where several devices are connected in parallel and each device has a unique SCSI ID for addressing.
- SCSI uses a controller (host adapter) that manages communication between CPU and all connected devices.
- When a device needs to transfer data, it requests the bus and a bus arbitration process decides which device gets control.
- After winning arbitration, the device becomes the bus master and performs data transfer using command, data, and status phases.
- SCSI supports high-speed synchronous or asynchronous data transfer and allows multiple devices to operate efficiently on one interface.
- It also supports advanced features like DMA, error detection, and command queuing for reliable performance.
- SCSI behaves like a multipoint external bus providing fast and flexible communication between computer and multiple peripherals.
- Generally, it can connect up to 16 peripheral devices on a single bus, which includes one host adapter.

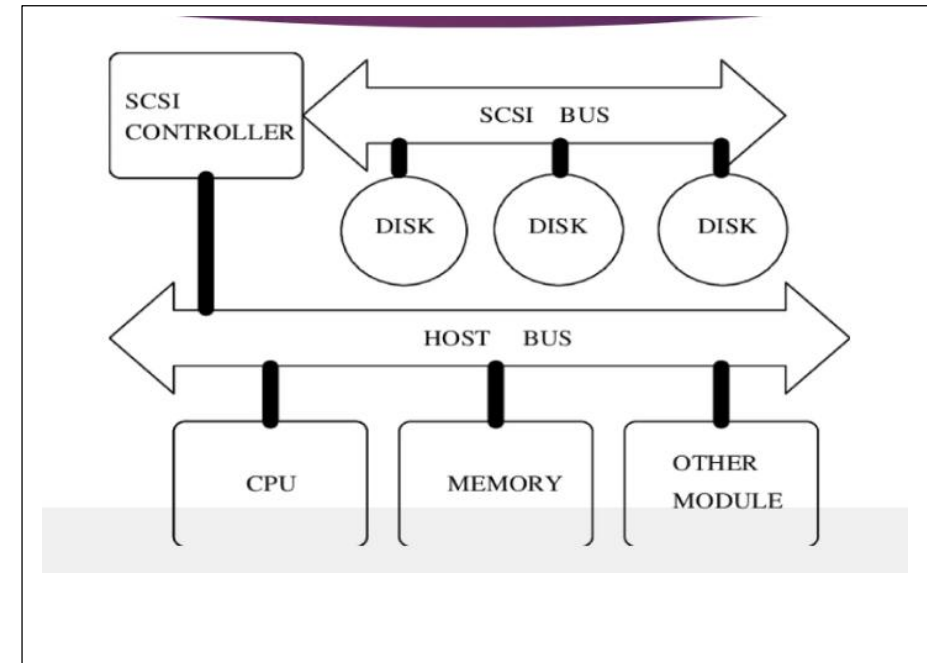
Hardware and Integration

- **Controller:** A dedicated controller manages data transfer between the devices and the SCSI bus.
- **Placement:** This controller is either embedded directly on the motherboard or added via a host adapter in an expansion slot.
- **SCSI BIOS:** The controller includes a small chip (BIOS) that provides the software needed to control the connected devices

The CPU sends a request for data through the Host Bus to the SCSI Controller.

The SCSI Controller identifies which specific Disk holds that data (using a unique SCSI ID).

The data travels from the Disk, across the SCSI Bus, through the Controller, and onto the Host Bus to be used by the Memory or CPU.



Input Output Processor (IOP)

An IOP is a processor that handles only the I/O processing.

The block diagram of a computer system with input output processor is shown in Figure 7.20. The computer system with I/O processor consists of three major components: (a) a memory unit (ii) the processor and (iii) the IOP. The IOP controls and manages the input-output tasks. On the command of CPU, the IOP can fetch, decode and execute I/O instructions which are loaded to perform the I/O operations.

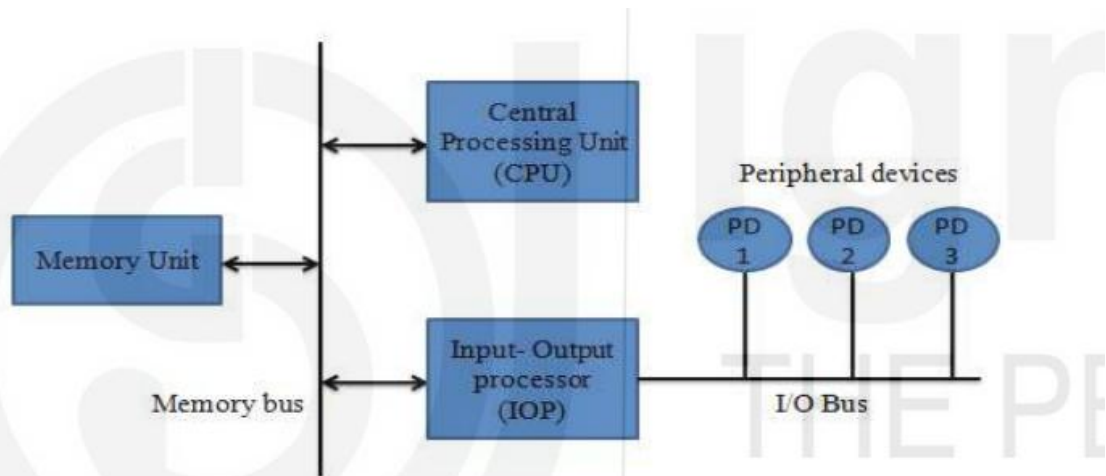


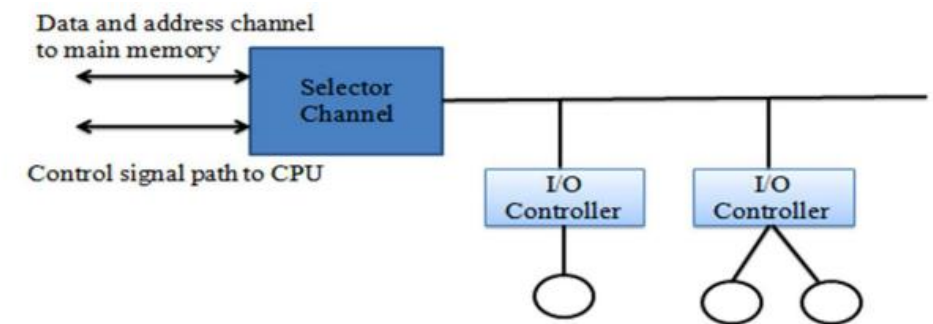
Figure 7.20: Block Diagram of a Computer System with I/O Processor

I/o Chanel and Its Characteristics

I/O Processor's Characteristics In simple words, an I/O processor (or I/O channel) is the extension of the concept of DMA technique.

An I/O channel holds complete control of the I/O operation and is also capable to execute I/O instructions. The CPU initiates the I/O operations and handovers the execution of I/O commands to I/O processor. In response, the I/O processor executes the I/O instructions stored into the main memory. The I/O channels can be divided into two categories:

- i. Selector I/O channels
- ii. Multiplexer I/O channels.



(a) Selector I/O Channel

A. Selector I/O channel: The selector I/O channel is a dedicated channel to perform data transfer exclusively with a high speed I/O device connected to it. Each I/O controller may control single or multiple I/O devices. Thus, I/O channels reduce the burden of the CPU by controlling the I/O controllers. The selector I/O channel is shown in

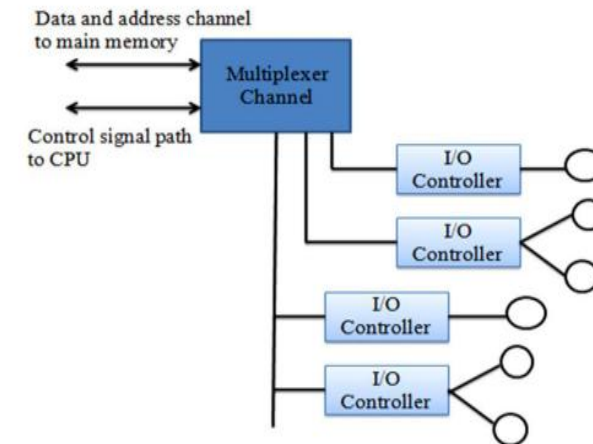
Figure 7.21(a).

B. Multiplexer I/O channel: Figure 7.21(b) depicts the multiplexer I/O channel. This I/O channel is capable to handle I/O operations with more than one I/O device altogether. For example-if, there are three slow devices interested to send the following bytes given as:

I st I/O device:	A1	A2	A3	A4	A5
II nd I/O device:	B1	B2	B3	B4	B5.....
III rd I/O device:	C1	C2	C3	C4	C5.....

Then, the byte multiplexer I/O channel may send the bytes as follows:
C1 B1 A1 C2 B2 A2 C3 B3 A3.....

For high-speed devices, block multiplexer I/O channels can be used which transfers data in form of blocks.



(b) Multiplexer I/O Channel

Figure 7.21: Structures of I/O Channels

Thank you